Understanding Air Pollutants and AQI in India from 2015-2020 Using a VAR Model

Mychelle Hale

California State University, Long Beach

ECON 586 – Econometrics II

Dr. Andre Harrison

Table of Contents

Abstract 3

Understanding Air Pollutants and AQI in India from 2015-2020 Using a VAR Model 4

Literature Review 4

Data and Definitions 5

Methodology. 8

Results. 10

Conclusion and Future Work. 10

Works Cited 14

Appendix 16

Footnotes 7

Tables 16

Figures 17

Abstract

In this paper we investigate the relationships between various pollutants and the air quality index (AQI) in India between 2015 and 2019. We aggregate the hourly observations from towers collecting pollution levels in 26 Indian cities to the weekly level in order to conduct our analysis. Through a time-series analysis, we identify patterns of seasonality and trend-cycles; then, we use a VAR model and SVAR model to predict the effects of various pollutants on AQI over time. We determine that there is Granger and instantaneous causality between carbon monoxide and the variables of interest and perform a historical decomposition.

Keywords: AQI, Pollutants, India, VAR, Time Series, SVAR, FEVD

Understanding Air Pollutants and AQI in India from 2015-2020 Using a VAR Model

India has experienced hazardous air quality for years. According to a published Q&A session with Dr. Sarath Guttikunda for the Centre for Policy Research (CPR), there are several pollutants that exceed the standard levels in India: $PM_{2.5}$, PM_{10}, nitrous oxide (NO_2), carbon monoxide (CO), and Ozone (O_3) (Guttikunda, 2017). The CPR also reports that although the main sources of pollution related to industrial activity, biomass burning, fossil fuels and operating vehicles contribute to poor air quality year-round, there are also seasonal contributions from fires, dust storms and salt near coastal areas. Additionally, the abundance of pollutants accelerates the effects of climate change and play a significant role in health issues related to the lungs, heart, and vision. The main contributors to $PM_{2.5}$ include carbon, sulfur dioxide (SO_2), and nitrates and secondary aerosols related to the transformations of chemical reactions and combinations of ($NO_x - CO - VOC$), and the consumption of ozone (O_3) related to the chemical reactions and combinations of ($NO_x - CO - VOC$). These will be defined in the Definitions section below. $PM_{2.5}$ is one of the main drivers in the calculation of the air quality index (AQI), so understanding the relationships between various pollutants and $PM_{2.5}$ is essential to understanding AQI.

In this paper, we use R to define a Vector Autoregressive (VAR) model to analyze the relationships between several pollutants and $PM_{2.5}$ and AQI in India, and how those relationships have evolved from January 2015 to July 2020. We also determine that there is both instantaneous and Granger causality between the pollutants and AQI.

Literature Review.

On Kaggle, the page where the data set can be accessed has links to notebooks for analyses conducted by others. One that particularly stood out was a data visualization by Parul

Pandey for the effects of the COVID-19 lockdown on the air quality in India (Pandey, 2020). In her analysis, Pandey provides visualizations for the cities with the highest levels of each pollutant. She then reports on the differences between the levels of various pollutants in 2019 compared to the start of the lockdown on March 26, 2020. As one might imagine, with a reduction in the number of vehicles on the road for almost a year, the pollutant levels drastically decreased. She also uses measurements from NASA's Tropospheric Monitoring Instrument (TROPOMI) to provide geographical visualizations for the near real-time comparisons of pollutant levels in 2019 and 2020. Monthly plots also indicate seasonality in the levels of each pollutant.

Another notebook, created by Ramshankar Yadhunath, aims to determine which cities to provide financial support to combat poor air quality. With five cities with the worst current AQIs on his list of cities to analyze, Yadhunath uses a probability matrix to express the probability that cities will transition from their AQI level to the next (Yadhunath, 2020). Furthermore, using the population and income per capita for the five cities of interest, Yadhunath determines that cities with smaller areas and lower numbers of children would most benefit from efforts to curb poor AQI. Higher numbers of children in a population indicate a growing population, so the cities with more children and larger areas will require more funding and greater efforts to curb pollution.

Data and Definitions

Data

The data set used in this analysis was made public by the Central Pollution Control Board in India and can be accessed on their website or on Kaggle (Kaggle, 2020). The data set spans the period between January 2015 to July 2020 and contains hourly and daily readings of

pollutant levels from stations in 24 cities in India, including some major cities like Delhi, Mumbai, and Kolkata.

Although readings for Benzene, Toluene, and Xylene, nitrogen oxide, (NO_x) and ammonia (NH_3) are included in the data set, they had very few observations, so we exclude them and focus the observations for $PM_{2.5}$, PM_{10}, SO_2, NO_2, and O_3. These pollutants are define in the Data and Definitions section below. Using Python, we aggregate the hourly data on a daily level and calculate the mean pollutant levels and AQI using the official AQI calculation (Rohanrao, 2020). Since the AQI and pollutant levels of cities close by are related to each other, rather than splitting the data up by city, we aggregate the observations on a weekly level and calculate the mean observation for each of the pollutants and AQI for each week in order to perform time series analysis. Since the data only contains observations for a five-year period, we aggregate at the weekly level rather than monthly to avoid the significant reduction of the number of observations. Estimating missing values is outside of the scope of this project since estimations would need to be done at the city level prior to aggregation. Some cities do not have daily observations, and this could be attributed to a lack of resources or issues with the measurement technologies. Estimation would require either k-Nearest Neighbors rather than forward or back filling missing observations. We would need to estimate observations based on the non-null observations from the city in question and surrounding cities, so we exclude the null observations from the daily and weekly aggregations.

Definitions

First, we will define some terms used throughout the analysis.

Air Quality Index (AQI) – Pandey defines Air Quality Index, or AQI, as being "used by government agencies to communicate to the public how polluted the air currently is or how

polluted it is forecast to become. There are six AQI categories, namely Good, Satisfactory, Moderately polluted, Poor, Very Poor, and Severe. The proposed AQI will consider eight pollutants (PM10, PM2.5, NO2, SO2, CO, O3, NH3, and Pb) for which short-term (up to 24-hourly averaging period) National Ambient Air Quality Standards are prescribed" (Pandey, 2020). Below is a visual representation of the AQI levels and their health impacts as reported by the Central Population Control Board (CCPB) in India.

AQI	Remark	Color Code	Possible Health Impacts
0-50	Good		Minimal impact
51-100	Satisfactory		Minor breathing discomfort to sensitive people
101-200	Moderate		Breathing discomfort to the people with lungs, asthma and heart diseases
201-300	Poor		Breathing discomfort to most people on prolonged exposure
301-400	Very Poor		Respiratory illness on prolonged exposure
401-500	Severe		Affects healthy people and seriously impacts those with existing diseases

Figure 1: Air Quality Index Levels and Health Impacts. Source: (Central Pollution Control Board, 2021)

Nitrous Oxide ($\boldsymbol{NO_x}$) – Nitrogen oxides are a group of seven gases and compounds composed of nitrogen and oxygen, sometimes collectively known as NO_xgases. The two most common and hazardous oxides of nitrogen are nitric oxide (NO) and nitrogen dioxide (NO_2) (Pandey, 2020).

Ozone ($\boldsymbol{O_3}$) - Ground-level ozone is a colorless and highly irritating gas that forms just above the earth's surface. It is called a "secondary" pollutant because it is produced when two primary pollutants react in sunlight and stagnant air. These two primary pollutants are nitrogen oxides (NO_x) and volatile organic compounds (VOCs) (Pandey, 2020).

$\boldsymbol{PM_{2.5}}$ – Tiny particulate matter, less than 2.5 μm in diameter, in the air that reduce visibility and cause the air to appear hazy when levels are elevated. These particles can travel deeply into the respiratory tract and blood stream, and irritate the eyes and respiratory tract; furthermore, they

can cause long term health issues including asthma, heart disease, chronic bronchitis, and lung cancer (New York Health Institute, 2021).

$\boldsymbol{PM_{10}}$ – Tiny particulate matter less than 10 μm in diameter. Exposure to high concentrations can result in health consequences like those of $PM_{2.5}$.

Sulfur Dioxide ($\boldsymbol{SO_2}$) – Sulfur dioxide, or SO2 is a colorless gas with a strong odor, similar to a just-struck match. It is formed when fuel containing sulfur, such as coal and oil, is burned, creating air pollution (Pandey, 2020).

Methodology

After aggregating the data on a weekly level, we obtain 237 observations for each of the previously listed variables. In Figure 2 of the

Figures section in the Appendix, we plot each of the time-series variables to identify any patterns of seasonality, constants, and trend-cycle. The plots for AQI, $PM_{2.5}, PM_{10}$ and CO show an overall decreasing trend-cycle while NO_2, SO_2 and ozone have polynomial trend-cycles. At first glance, we can identify patterns of seasonality in all the pollutants. These visual identifications align with the information reported in Dr. Guttikunda's Q&A session.

Since the plots indicated seasonality, we remove the seasonal component by seasonally adjusting each of the variables before running the Augmented Dickey Fuller (ADF) Test to test for stationarity. The null hypothesis of the ADF Test indicates the lack of a unit root while the alternative indicates that a unit root is present. We determine that CO and SO_2 are the only pollutants that are stationary after seasonally adjusting. The exact results of these ADF Tests are reported in Table 3 of Tables section below. Next, we take the first difference of the non-stationary seasonally adjusted variables to detrend them. The ADF Tests for the detrended seasonally adjusted variables indicate that all variables are now stationary. We plot the detrended seasonally adjusted time-series variables again to identify any signs of high volatility. The results of the time-series plots for the seasonally adjusted detrended variables are as follows. AQI levels were highly volatile for the first quarter of 2015, the last quarter of 2017 and in the beginning and end of 2019. Between these periods of high volatility, there were mostly periods of low fluctuations in volatility except in 2018 when fluctuations were moderate. $PM_{2.5}$ had mild fluctuations in volatility between 2015 to mid-2016, and from 2018 to 2020. There was also a period of high fluctuations in volatility between late-2016 to 2018. PM_{10} levels had large fluctuations between 2015 to 2016. Then, after 2016, PM_{10} levels were mildly volatile. NO_2

levels were highly volatile throughout the period between 2015 to 2018, but after 2018, there were only mild fluctuations in variability.

Since all the detrended variables pass the ADF Test, we move forward with our analysis. We combine all the detrended variables into a new data frame. Then, we plot the ACF up to 100 lags for the AQI to determine which lags are significant. The plot shows that lags 2, 40, 57 and 71 are significant. Although there are many significant lags in the middle of the detrended AQI, we will set the maximum number of lags used for analysis to eight for simplicity and to avoid issues related to dimensionality. Next, we use the *VARselect* command in the *vars* package in R to find the optimal number of lags for our VAR model. Using the Akaike Information Criterion (AIC) and the Schwartz Information Criterion (SIC) we determine which VAR(p) model, where p indicates the number of lags, is optimal for analysis. As indicated in the Table 1below, the AIC selects Lag 2 as our optimal model while the SIC favors Lag 1.

Table 1: Model Selection Criteria for Lags

	Lag 1	**Lag 2**	**Lag 3**	**Lag 4**	**Lag 5**	**Lag 6**	**Lag 7**	**Lag 8**
AIC	20.769	**20.335**	20.353	20.445	20.470	20.450	20.420	20.364
SIC	**21.406**	21.611	22.266	22.996	23.658	24.276	24.884	25.466

The SIC tends to favor the more parsimonious model. Since we are analyzing weekly data, one or two lags does not seem sufficient for our analysis. Although we enter 7 lags and a lag max of 8 in our VAR model on the detrended data set, the model selects two lags. The VAR(2) model can be specified as:

$$AQI_t = \underset{0.097}{-0.059}\,AQI_{t-1} - \underset{0.146}{0.420}\,PM_{2.5_{t-1}} + \underset{0.552}{0.049}\,PM_{10_{t-1}} - \underset{0.419}{0.731}\,NO_{2_{t-1}} - \underset{1.865}{9.181}\,CO_{t-1} - \underset{0.580}{3.834}\,SO_{2_{t-1}} - \underset{0.357}{0.005}\,O_{3_{t-1}}$$
$$- \underset{0.090}{0.270}\,AQI_{t-2} - \underset{0.139}{0.181}\,PM_{2.5_{t-2}} + \underset{0.055}{0.047}\,PM_{10_{t-2}} - \underset{0.422}{0.289}\,NO_{2_{t-2}} + \underset{1.728}{0.022}\,CO_{t-2} + \underset{0.600}{0.427}\,SO_{2_{t-2}} - \underset{0.357}{0.529}\,O_{3_{t-2}}$$

Next, we define the $\hat{\beta}$ matrix using the VAR(2) model estimated in the previous step. We plug the VAR(2) model into the SVAR function. This allows us to use the Structural Vector Autoregression (SVAR) to directly estimate the $\hat{\beta}$ matrix. For simplicity, we use recursive estimation to estimate the lower triangle of the $\hat{\beta}$ matrix. The results are reported in the along with the results of the Forecast Error Variance Decomposition (FEVD) in the **Results** section below.

Results

After creating and storing our VAR(2) model, we check for Granger and instantaneous causality between the AQI and the other variables. The test for Granger causality has a null hypothesis that carbon monoxide Granger causes the other variables. This results in a p-value less than 0.001 and a corresponding F-test statistic of 3.556. As a result, we reject the null hypothesis at every significance level. This means that changes to carbon monoxide in time t result in better predictions for future changes to the other variables. We also test for instantaneous causality using the null hypothesis that there is no instantaneous causality between carbon monoxide and the other variables. This resulted in a $\chi_6^2 = 84.751$, and a corresponding p-value less than 0.001, so we reject the null hypothesis at every significance level that there is no instantaneous causality between AQI and the other variables. This means that changes to carbon monoxide in time t result in better predictions for the other variables in the same time step.

After creating our VAR(2) model we estimate our $\hat{\beta}$ matrix, which is specified as

$$\hat{\beta} = \begin{bmatrix} 1.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 \\ 0.294 & 1.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 \\ 0.427 & 0.240 & 1.000 & 0.000 & 0.000 & 0.000 & 0.000 \\ 0.061 & 0.046 & 0.037 & 1.000 & 0.000 & 0.000 & 0.000 \\ 0.190 & 0.179 & 0.045 & 0.137 & 1.000 & 0.000 & 0.000 \\ 0.009 & -0.023 & -0.017 & -0.152 & 0.003 & 1.000 & 0.000 \\ 0.061 & -0.007 & 0.048 & 0.281 & 0.281 & 0.135 & 1.000 \end{bmatrix}.$$

Then, we check if the model is sufficient to model the data set using the *MarchTest* on the standardized residuals. We observe both the Portmanteau and Rank Based Test which both hypothesize that there is no conditional heteroskedasticity among the standardized residuals of the VAR(2). Both tests result in p-values that are 0 and corresponding test-statistics of 847.170 (Portmanteau) and 195.280 (Rank Based). We also visually test for conditional heteroskedasticity among the standardized residuals of the modeled AQI of the VAR(2) model using diagnostic plots. Although there are a few large spikes around 2015 and 2019 in the plot of the standardized residuals, the variance is otherwise constant. The PACF indicates that lags 10, 11 and 14 are significant, but these were not included in our model. This would be good to keep in mind for future analysis if we wanted to expand the number of lags included in our model and address issues with dimensionality. The QQ-plot for the squared residuals indicates that the normal distribution may not be sufficient for modeling the tails of the residuals. We see some outliers on both tails, but most of the observations appear to be within the confidence bands. The PACF does not indicate that there are any significant lags up to 20 lags. As a result of the diagnostic plots and the MarchTest, we determine that the model is sufficient to model the data.

The historical decomposition of the detrended AQI on a weekly level did not provide much insight into which pollutant contributed most to the AQI over time. It seems that shocks to $PM_{2.5}$ had a significant impact on the AQI up to 2017, but shocks to AQI had the biggest impact on the fluctuations in AQI overall. Shocks to carbon monoxide also had an apparent effect on the AQI; however, the shocks of the other variables mostly blended, so the plot was not very useful. The historical decomposition can be found in Figure 5 of the Figure section.

Using the Forecast Error Variance Decomposition (FEVD) from the SVAR model, we determine how the variability of the AQI is impacted by the variation in each variable 10 periods ahead. The output of the is included in the table below.

Table 2: AQI FEVD 10 Lags Ahead

AQI	$PM_{2.5}$	PM_{10}	NO_2	CO	SO_2	O_3
1.000	0.000	0.000	0.000	0.000	0.000	0.000
0.041	0.041	0.002	0.057	0.731	0.128	0.000
0.034	0.039	0.002	0.052	0.758	0.113	0.002
0.034	0.039	0.002	0.056	0.747	0.119	0.003
0.033	0.039	0.002	0.056	0.753	0.114	0.003
0.032	0.039	0.002	0.055	0.754	0.114	0.003
0.032	0.039	0.002	0.056	0.753	0.114	0.003
0.032	0.039	0.002	0.056	0.753	0.114	0.003
0.032	0.039	0.002	0.056	0.753	0.114	0.003
0.032	0.039	0.002	0.056	0.753	0.114	0.003

Conclusion and Future Work

The VAR(2) model is sufficient for modeling the detrended pollution and AQI data set. As expected, the air quality in India is most impacted by carbon monoxide levels since carbon contributes significantly to $PM_{2.5}$ which is a major driver of the AQI calculation. COVID-19 has significantly reduced pollution levels over the last year, and India's air quality issues are on the radar of several governments around the world. People and governments around the world are actively searching for ways to reduce pollution in India to slow down the effects of climate change and provide Indians with healthier air. Not only is healthier air beneficial to the overall health of Indians, but it is also necessary to survive a virus that aggressively attacks the respiratory system. With fewer vehicles on the road due to lockdown, there is less dust being kicked up, less diesel being used and emitted and less industrial activity. One way to help with

the issue related to dust being displaced by vehicles would be to invest in infrastructure to build paved roads in areas where this is an issue.

With more time, we could also include the analysis of the IRFs for each of the variables. Although the plots were created, due to limitations on space and time, we could not include the analysis in this report. In the future, we could expand this analysis to include more pollutants and data from more locations around the world. It would be interesting to compare the pollution levels and air quality between countries whose air qualities are not apparently related. Specifically, analysis of the air qualities and pollution levels of Brazil and countries in northeastern Africa would be great for a time-series analysis. At NASA Jet Propulsion Lab (JPL), we are working on a project to determine how the pollutants resulting from biomass burning in Brazil and northeastern Africa are transported across the Atlantic Ocean by wind currents. We are hoping to identify a direct relationship between ammonia levels in Africa and nitrogen levels in Brazil and vice versa. On a map their relationship seems apparent, but we are hoping to provide evidence using a model. A time-series analysis would allow us to analyze the relationships on a weekly or monthly level.

Works Cited

Central Pollution Control Board. (2021, May). *National Air Quality Index*. Retrieved from Central Pollution Control Board: https://app.cpcbccr.com/AQI_India/

Environmental Protection Agency. (2020, June 30). *What is Ozone? | Ozone and Your Patients' Health | US EPA*. Retrieved from Environmental Protection Agency: https://www.epa.gov/ozone-pollution-and-your-patients-health/what-ozone

Guttikunda, S. (2017, December 14). *Air pollution in Indian cities: Understanding the causes and the knowledge gaps*. Retrieved from Air pollution in Indian cities: Understanding the causes and the knowledge gaps: https://www.cprindia.org/news/6569

Kaggle. (2020, July 28). *Air Quality Data in India (2015 - 2020)*. Retrieved from Kaggle: https://www.kaggle.com/rohanrao/air-quality-data-in-india

New York Health Institute. (2021, May 11). *Fine Particles (PM 2.5) Questions and Answers*. Retrieved from Department of Health: https://www.health.ny.gov/environmental/indoors/air/pmq_a.htm#:~:text=PM2.5%20are%20tiny%20particles,no%20wind%20or%20air%20mixing.

Pandey, P. (2020, May 25). *Breathe India: COVID-19 effect on Pollution*. Retrieved from Kaggle: https://www.kaggle.com/parulpandey/breathe-india-covid-19-effect-on-pollution

Rohanrao, V. (2020, August 22). *Calculating AQI (Air Quality Index) Tutorial*. Retrieved from Kaggle: https://www.kaggle.com/rohanrao/calculating-aqi-air-quality-index-tutorial?scriptVersionId=41199538

Yadhunath, R. (2020, September 11). *Where to Invest to Combat Air Pollution in India*. Retrieved from Kaggle: https://www.kaggle.com/thedatabeast/where-to-invest-to-combat-air-pollution-in-india

Appendix

Tables

Table 3: Results of ADF Test on Seasonally Adjusted Data Set

Variable	Test Statistic	p-value
AQI	-1.316	0.197
$PM_{2.5}$	-1.509	0.135
PM_{10}	-1.719	0.085
NO_2	-0.667	0.403
CO	-3.209	< 0.01
SO_2	-2.242	0.024
O_3	-0.424	0.481

Figures

Figure 2.

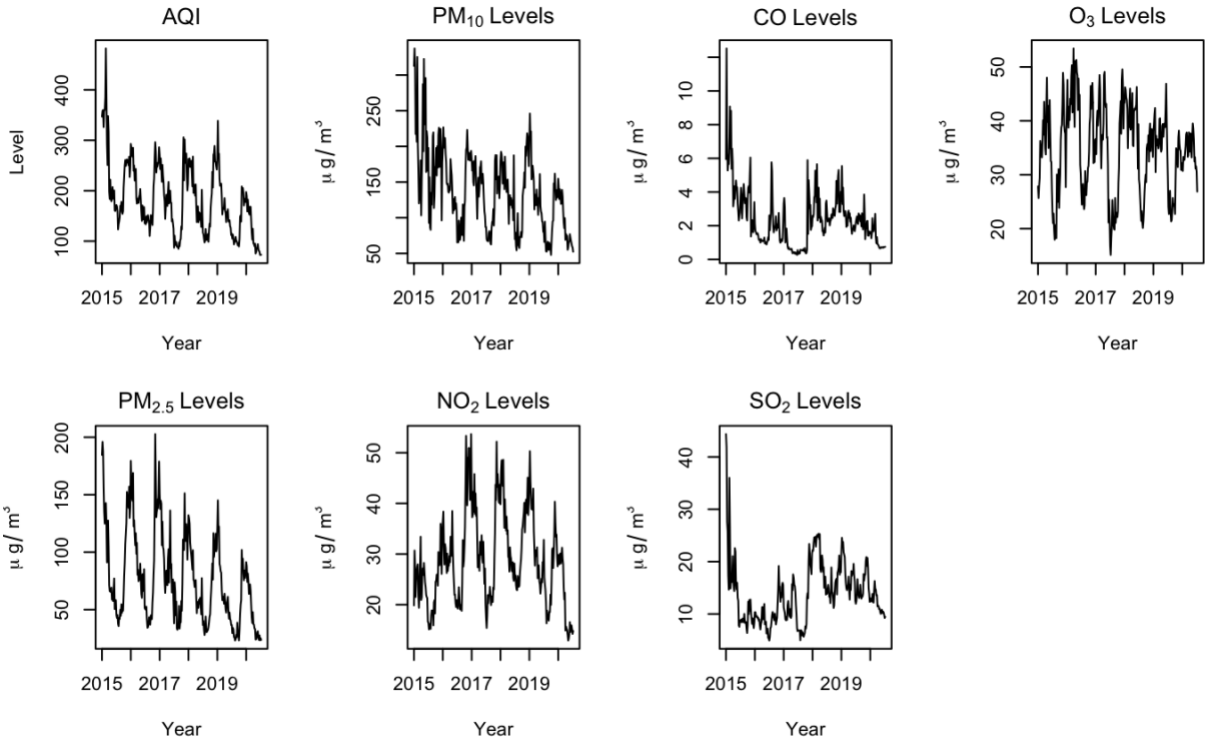


Figure 2: Time-Series Plots for Pollutants and AQI

Figure 3.

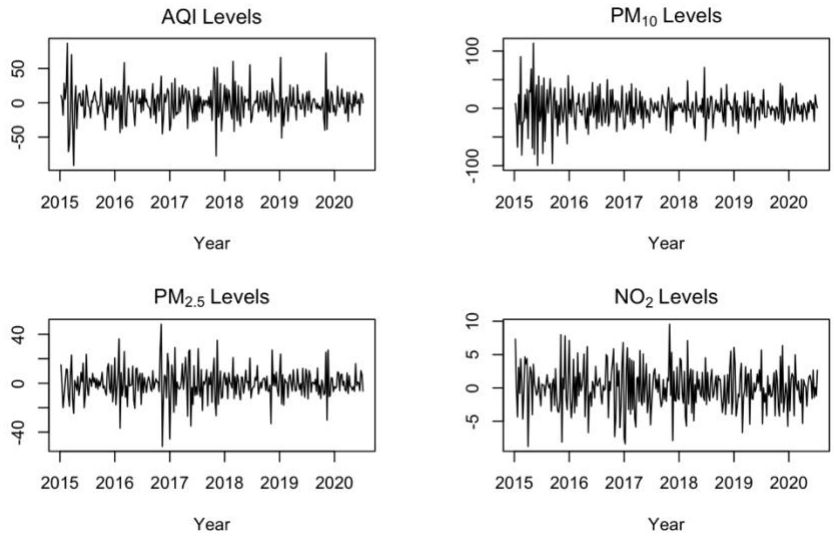


Figure 3: Time-Series Plot for Seasonally Adjusted Detrended Variables

Figure 4.

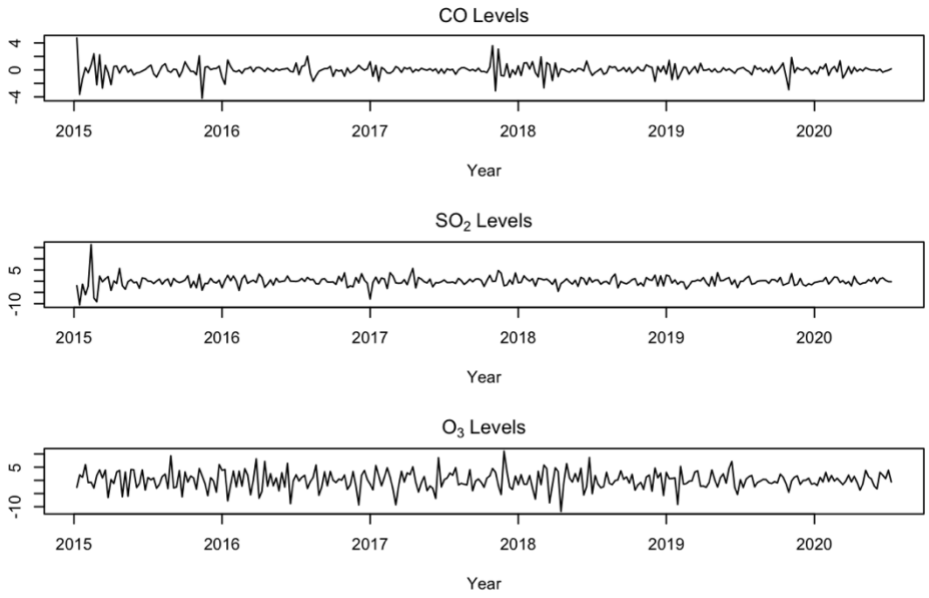


Figure 4: Time-Series Plot for Seasonally Adjusted Detrended Variables

Figure 5.

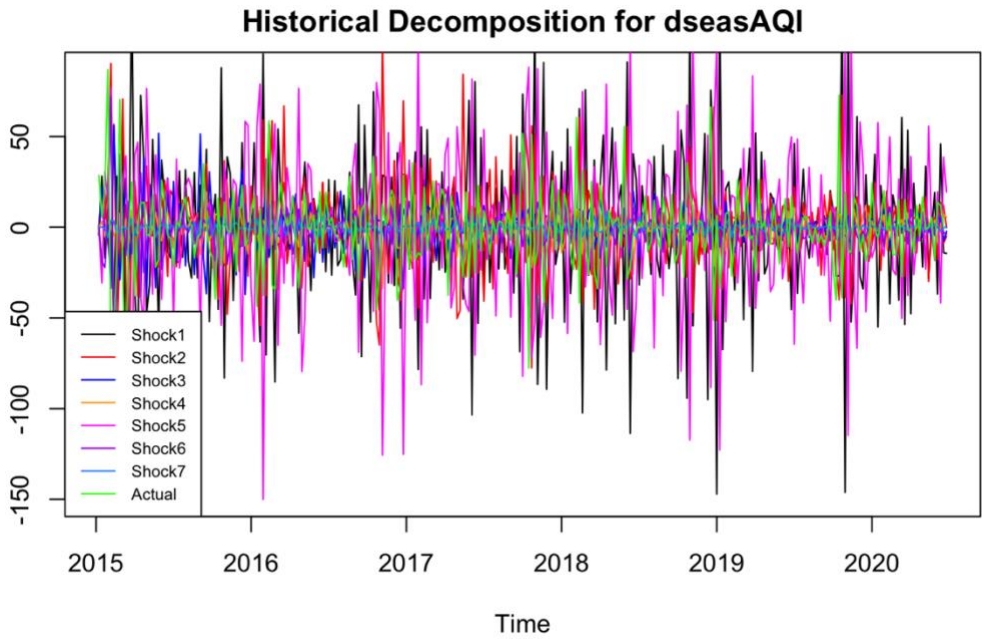


Figure 5: Historical Decomposition of Each Detrended Variable

R Code

To run this code, you will need to have the *rmarkdown* package installed in R.

```
1.
2.
3.  ```{r setup, include=FALSE, warning = FALSE}
4.  knitr::opts_chunk$set(echo = F,
5.                        fig.align   = 'center',
6.                        tidy.opts=list(width.cutoff=60), dev = 'pdf'
7.                        ,tidy=TRUE, include = T, results = 'hide', warning = F, fig.show
    ='hide' )
8.  ```
9.
10. ```{r setuppackages, results = 'hide', warning = FALSE, include = FALSE}
11. require(knitr)
12. require(rmarkdown)
13. require(sandwich)
14. require(fUnitRoots)
15. require(ggplot2)
16. require(forecast)
17. require(vars)
18. require(urca)
19. library(TSstudio)
20. require(dplyr)
21. library(imputeTS)
22. library(tidyverse)
23. require(lubridate)
24.
25. ```
```

````
26.
27.
28. # R Code
29.
30. ```{r}
31. city_day = read.csv('df_city_day.csv', header = T)
32. # city_day = na.omit(city_day)
33. stationscities = read.csv('df_station_day.csv', header = T)
34.
35. # stations = read.csv('stations.csv', header = T)
36. # citylocs = read.csv('IndianCitiesDatabase.csv')
37.
38. ```
39.
40. ```{r}
41. dfday = read.csv('df_day.csv', header = T)
42. dfday['BTX'] = dfday['Benzene']+dfday['Toluene']+dfday['Xylene']
43. dfday['ParticulateMatter'] = dfday['PM2.5']+dfday['PM10']
44. # dfcols = c('Date', 'AQI','PM2.5','PM10','NO2', 'CO', 'SO2','O3', 'BTX',
    'ParticulateMatter')
45. dfcols = c('Date', 'AQI','PM2.5','PM10','NO2', 'CO', 'SO2','O3')
46. dfdays = dfday[dfcols]
47.
48. ```
49.
50. #Picking top two cities with worst AQI
51.
52. ```{r}
53.
54. dfdays <- dfdays %>%
55.   group_by(week = factor_aweek(date2week(Date))) %>%
56.   mutate(AQI.wk.average = mean(AQI))%>%
57.   ungroup() %>%
58.   group_by(month = format(as.Date(Date), "%Y-%m")) %>%
59.   mutate(AQI.mo.average = mean(AQI))
60.
61.
62. dfweeks = aggregate(dfdays[,-c(1,9,10,11,12)], list(dfdays$week), mean)
63. names(dfweeks)[1] = paste("Week")
64.
65. dfmonths = aggregate(dfdays[,-c(1,9,10,11,12)], list(dfdays$month), mean)
66. names(dfmonths)[1] = paste("Month")
67.
68. #for forecasting. We will choose the week that corresponds to this date. then select it from
    dfweeks
69. loc = which(dfdays[,1]=="2019-01-01")
70.
71. CO1 <- dfdays[(1:loc),2]
72.
73.
74. AQI = dfweeks$AQI
75. AQIWeek = dfweeks$AQI.wk.average
76. # AQIMonth = dfmonths$AQI.mo.average
77. PM25 = dfweeks$PM2.5
78. PM10 = dfweeks$PM10
79. NO2 = dfweeks$NO2
80. CO = dfweeks$CO
81. SO2 = dfweeks$SO2
82. O3 = dfweeks$O3
83.
84. ```
85.
86. ```{r Plots}
87. par(mfcol=c(2,4))
88. plot(ts(dfweeks[,2], start = c(2015, 1), freq = 52),
````

```
      main = expression(AQI), xlab = "Year", ylab = "Level")
plot(ts(dfweeks[,3], start = c(2015, 1), freq = 52),
      main = expression(PM[2.5]~Levels), xlab = "Year", ylab = expression(mu~"g"/~m^3))
plot(ts(dfweeks[,4], start = c(2015, 1), freq = 52),
      main = expression(PM[10]~Levels), xlab = "Year", ylab = expression(mu~"g"/~m^3))
plot(ts(dfweeks[,5], start = c(2015, 1), freq = 52),
      main = expression(NO[2]~"Levels"), xlab = "Year", ylab = expression(mu~"g"/~m^3))
plot(ts(dfweeks[,6], start = c(2015, 1), freq = 52),
      main = expression(CO~"Levels"), xlab = "Year", ylab = expression(mu~"g"/~m^3))
plot(ts(dfweeks[,7], start = c(2015, 1), freq = 52),
      main = expression(SO[2]~"Levels"), xlab = "Year", ylab = expression(mu~"g"/~m^3))
 plot(ts(dfweeks[,8], start = c(2015, 1), freq = 52),
       main = expression(O[3]~"Levels"), xlab = "Year", ylab = expression(mu~"g"/~m^3))


 #ADF Test: H0 = unit root present. All are stationary when you do ADF with ct except NO2
 adfTest(AQI, lags = 1, type = 'nc') # unit root at 95. non-stationary
 # adfTest(AQIWeek, lags = 1, type = 'ct') #stationary
 # adfTest(AQIMonth, lags = 1, type = 'ct') #stationary
 adfTest(PM25, lags = 1, type = 'nc') # no unit root at 95. stationary
 adfTest(PM10, lags = 1, type = 'nc') #no unit root at 95. stationary
 adfTest(NO2, lags = 1, type = 'nc')# not stationary at any level
 adfTest(CO, lags = 1, type = 'nc')#no unit root. Stationary
 adfTest(SO2, lags = 1, type = 'nc')# stationary. no unit root at 95
 adfTest(O3, lags = 1, type = 'nc')#not stationary. unit root @ all levels

 ## run a regression with time trend to identify the significance of the time trend

 ```
 ```{r SeasonPlots}
 par(mfrow=c(3,2))
 decompAQI = stl(ts(AQI, start = c(2015, 1), freq = 52), s.window = 9)
 plot(decompAQI, main = "STL Decomposition: AQI") #downward trend and seasonality

 decomp25 = stl(ts(dfweeks[,3], start = c(2015, 1), freq = 52), s.window = 9)
 plot(decomp25, main = "STL Decomposition: PM 2.5") #downward trend and seasonality

 decomp10 = stl(ts(dfweeks[,4], start = c(2015, 1), freq = 52), s.window = 9)
 plot(decomp10, main = "STL Decomposition: PM 10") #downward trend and seasonality

 decompNO2 = stl(ts(dfweeks[,5], start = c(2015, 1), freq = 52), s.window = 9)
 plot(decompNO2, main = "STL Decomposition: NO2") #no clear trend. Seasonality

 decompCO = stl(ts(dfweeks[,6], start = c(2015, 1), freq = 52), s.window = 9)
 plot(decompCO, main = "STL Decomposition: Carbon Monoxide") #Downward trend and
   seasonality

 decompSO2 = stl(ts(dfweeks[,7], start = c(2015, 1), freq = 52), s.window = 9)
 plot(decompSO2, main = "STL Decomposition: Sulfur Dioxide Levels") #unsure about this
   trend. No trend? seaosonality

 decompO3 = stl(ts(dfweeks[,8], start = c(2015, 1), freq = 52), s.window = 9)
 plot(decompO3, main = "STL Decomposition: Ozone") #no trend

 ```
 ```{r}
 ##dont need to difference the values that are already stationary.
 ## remove the number of rows = differences from the parameters that are already
   stationary, then cbind all of them together. See HW 7
 ## then use VAR Select to determine the number of optimal lags for the binded data set
 # then 3 and 4 from email. !!!!!! *********!!
 seasAQI = seasadj(decompAQI)
 seasPM25 = seasadj(decomp25)
 seasPM10 = seasadj(decomp10)
 seasNO2 = seasadj(decompNO2)
```

```
seasCO = seasadj(decompCO)
seasSO2 = seasadj(decompSO2)
seasO3 = seasadj(decompO3)

#check if they are stationary now.
adfTest(seasAQI, lags = 1, type = 'nc') # unit root at 95. non-stationary
# adfTest(AQIWeek, lags = 1, type = 'nc') #stationary
# adfTest(AQIMonth, lags = 1, type = 'nc') #stationary
adfTest(seasPM25, lags = 1, type = 'nc') # no unit root at 95. stationary
adfTest(seasPM10, lags = 1, type = 'nc') #no unit root at 95. stationary
adfTest(seasNO2, lags = 1, type = 'nc')# not stationary at any level
adfTest(seasCO, lags = 1, type = 'nc')#no unit root. Stationary
adfTest(seasSO2, lags = 1, type = 'nc')# stationary. no unit root at 95
adfTest(seasO3, lags = 1, type = 'nc')#not stationary. unit root @ all levels

```
```{r}
#
par(mfcol = c(2,2))
dseasAQI = diff(seasAQI, lag = 1, differences = 1)
plot(ts(dseasAQI, start = c(2015, 2), freq = 52),
     main = expression(AQI~Levels), xlab = "Year", ylab = "")
#
dseasPM25 = diff(seasPM25, lag = 1, differences = 1)
plot(ts(dseasPM25, start = c(2015, 2), freq = 52),
     main = expression(PM[2.5]~Levels), xlab = "Year", ylab = "")
#
dseasPM10 = diff(seasPM10, lag = 1, differences = 1)
plot(ts(dseasPM10, start = c(2015, 2), freq = 52),
     main = expression(PM[10]~Levels), xlab = "Year", ylab = "")
#
dseasNO2 = diff(seasNO2, lag = 1, differences = 1)
plot(ts(dseasNO2, start = c(2015, 2), freq = 52),
     main = expression(NO[2]~Levels), xlab = "Year", ylab = "")
#
par(mfcol = c(3,1))
dseasCO = diff(seasCO, lag = 1, differences = 1)
plot(ts(dseasCO, start = c(2015, 2), freq = 52),
     main = expression(CO~Levels), xlab = "Year", ylab = "")
#
dseasSO2 = diff(seasSO2, lag = 1, differences = 1)
plot(ts(dseasSO2, start = c(2015, 2), freq = 52),
     main = expression(SO[2]~Levels), xlab = "Year", ylab = "")
#
dseasO3 = diff(seasO3, lag = 1, differences = 1)
plot(ts(dseasO3, start = c(2015, 2), freq = 52),
     main = expression(O[3]~Levels), xlab = "Year", ylab = "")
#
```

```{r}
#check if they are stationary now.
adfTest(dseasAQI, lags = 1, type = 'ct') #unit root at 95. non-stationary
# adfTest(AQIWeek, lags = 1, type = 'ct') #stationary
# adfTest(AQIMonth, lags = 1, type = 'ct') #stationary
adfTest(dseasPM25, lags = 1, type = 'ct') #no unit root at 95. stationary
adfTest(dseasPM10, lags = 1, type = 'ct') #no unit root at 95. stationary
adfTest(dseasNO2, lags = 1, type = 'ct') #not stationary at any level
adfTest(dseasCO, lags = 1, type = 'ct')#no unit root. Stationary
adfTest(dseasSO2, lags = 1, type = 'ct') #stationary. no unit root at 95
adfTest(dseasO3, lags = 1, type = 'ct')#not stationary. unit root @ all levels
acf(dseasAQI, lag.max = 100)
```

```
# VAR Select
```

````
```{r}
## create a new data frame with differenced variables

daseas = cbind(dseasAQI,dseasPM25,dseasPM10, dseasNO2,dseasCO, dseasSO2, dseasO3)

vseseas = VARselect(y=daseas,lag.max = 8, type="none") #no constant/time trend based on ?


crit = vseseas$criteria

seq = c(1:3)
seqs = c(1:8)
crit1 = matrix(NA, 3, 8,
            dimnames = list(c("AIC", "HQC","SIC"), paste("Lag ", seqs,
                                                         sep =""")))
for (i in seq){
  for (j in seqs){
    crit1[which(seq==i),which(seqs==j)] = crit[i,j]
  }
}

kable(round(crit1,3), caption = "Model Selection Criteria for Lags")

```

```{r}
#identifying the optimal lag length, selection based on criteria
selseas  = matrix(NA, 3, 1,
            dimnames = list(c("AIC", "HQ","SIC"), paste("Lag",
                                                         sep =""")))
sel = round(vseseas$selection,3) #tells you which one has the smallest from each model selection
for (i in seq){
  selseas[which(seq==i)] = sel[i]
}
kable(selseas, caption = "Optimal Lag Lengths") #SIC is more parsimonious while AIC likes larger models better.

```

```{r}
library(vars)
#just checking the difference in output. There is not one.
p = 2 #optimal lag based on previous question
#run reduced form VAR with lag length = p, and w/o a constant
va = vars::VAR(daseas,p=p,lag.max = 8,type="none")
summary(va)

p2 = 7 #optimal lag based on previous question
va2 = vars::VAR(daseas,p=7,lag.max =8,type="none")
summary(va2)
```

```{r}
bmat = diag(7) #question specifies 4x4. If this was not given, we would use the number of betas in the equation. This is the number of variables included in the equation. If there are 4 vars, then 4x4. Takes away the covariation between the variables.

diag(bmat) = 1

#define the diagonal and create our NA values in the matrix
#Identify the NAs we plan to solve for
# for setting up the Bmatrix: upper diagonal is all 0, lower diagonal is all NAs, diagonal is all NA.
````
````

```
bmat[2,1] = NA
bmat[3,1:2] = NA
bmat[4,1:3] = NA
bmat[5,1:4] = NA
bmat[6,1:5] = NA
bmat[7,1:6] = NA


#Next we solve for the system to get our NA values for the #B-matrix
#SVAR command to run regression to solve unknowns
#we use the reduced form results/data
#estimation type is direct
#then we specify our A-matrix and B-matrix
#warning bc number of meqns is = to the # of unknowns, so we cannot perform testing
# reg = SVAR(va, estmethod = 'direct', Amat = NULL, Bmat = bmat)
reg = vars::SVAR(va,estmethod = "direct",Amat = NULL, Bmat=bmat)
  #estmethod is determined by the Bmatrix. If we do it with NA assigned instead of random, then we are doing recursive identification which is direct. Lecture 9, Slide 13-15.

  #estmethod is determined by the Bmatrix. If we do it with NAs random, then we are doing non-recursive identification which is direct. Lecture 9 Slide 16

#Amatrix: we don't need to do this but look at the lecture also

#Lecture 9, slide 25-29: recursive restrictions/identification example

kable(round(reg$B, 3))

```

```{r}
vasum <- summary(va)
vacovres = vasum$covres
causality(va, cause = "dseasCO")
vacovres
vacovres%*%t(vacovres)

```

Testing Residuals

Serial Correlation

```{r}
resi = residuals(va, standardize = T)
MarchTest(resi) #idk if this is right


#do this for each one
adfTest(resi[,1], lag = 1) # no serial correlation
adfTest(resi[,2], lag = 1)

#test for conditional heterskedasticity. Do this for each variable.
par(mfrow=c(2,2))
plot(ts(resi[,1], start = c(2015,1), freq = 52),
     xlab = "Year", ylab = "stand-resi",
     main = "Standardized Residuals")
acf(resi[,1], lag = 20, main = "ACF for Residuals")
qqPlot(resi[,1], distribution = "norm", main = "QQ-Plot for Residuals", ylab = "resi")
pacf(resi[,1]^2, lag = 20, main = "PACF for Squared Residuals")
```

IRF

````
```{r}
N = ncol(daseas)
nam = colnames(daseas)
#Next we create a loop for the impulse response's functions for each period from 0 to 20
#and we plot these outcomes for ffr
for (i in 1:N) {
  IRF = irf(reg,impulse="dseasAQI",response=nam[i],n.ahead=10,
              ortho = TRUE, boot=TRUE, ci=0.95)
  plot(IRF)
}

```

## FEVD

```{r}
FEVDseasAQI = fevd(reg, n.ahead = 10)$dseasAQI #variable of interest was specified in the question. FEVD = forecast error variance decomposition.

  #all the results explain the variation of the exchange rate based off shocks to any one of these variables.
kable(round(FEVDseasAQI,3), caption = 'Exchange Rate Forecast Error Variance Decomposition 10 Lags')

```
# Historical Decomposition

```{r 1b}

N = ncol(daseas)
nam = colnames(daseas)

#Step 1: first obtain companion form. First obtain structural shocks using regression from the b matrix.

#Step 1a
B = reg$B #obtain our B matrix from the structural VAR model

#Step 1b. residuals from data set from VAR using updated da data using optimal lag length
u = residuals(va) #obtain the residuals from the reduced form model

#Step 1c. transpose B matrix and multiply by residuals
w = solve(B)%*%t(u) #calculate the structural shocks as
                    #B inverse times our residual
w = t(w)

### Step 2: Next calculate thetas. Obtain companion form with 2 lags. Lags were specified in the VAR. N is the number of columns in the data set.  companion form is like B matrix, but we are interested in lags rather than number of vars.

## if we were interested in 2 lags, we would do 2x2. 6 lags, 6x6

B = reg$B #obtain our B matrix from the structural VAR model
u = residuals(va) #obtain the residuals from the reduced form model
w = solve(B)%*%t(u) #calculate the structural shocks as
                    #B inverse times our residual
w = t(w)

### Next calculate thetas
#first obtain companion form

a = Bcoef(va) #va is the var model we specified earlier. dimensions are number of columns times number of variables to determine dimensions of the matrix. 3 vars, 4 lags = 12 rows, 12 cols
a
````
````

```
396.  I = diag(N) # N is number of columns of the da dataset (number of variables)
397.  oh = matrix(0,N,N)
398.  ```
399.
400.  ```{r}
401.  # A = rbind(a, cbind(I, oh))
402.
403.  # 3 lags: A = rbind(a,cbind(I, oh, oh))
404.  ## A = rbind(a, cbind(oh, I, oh))
405.
406.  #if we were interested in more lags with more variables, we need to shift both A matrix
    and J matrix.
407.
408.
409.  # Note: when you use the square matrix, then you can do Cholesky decomp form. 5:04 pm
    about 1.5 hours into the discussion. Ask Tien about this
410.
411.
412.  #next obtain J matrix
413.  J = cbind(I, oh)
414.  #3 lags: J = cbind(I,oh, oh)
415.
416.
417.  # 6 vars, 3lags
418.  A <- Bcoef(va)
419.
420.  id <- diag(7) #number of vars
421.  zeros <- matrix(0,nrow=7,ncol=7) # row and col is number of vars
422.  An <- rbind(A,cbind(id,zeros))
423.  ```
424.  ```{r}
425.  ## creates a list to generate list of A from 0 to t-1
426.  An.list = list()
427.  An.list[[1]] = as.matrix(An)
428.  TT = length(w[,1])
429.
430.  for (j in 1:(TT-1)){
431.    An.list[[j+1]] = An.list[[j]]%*%An.list[[1]]
432.  }
433.
434.  ## In this case Theta 1 is the Cholesky decomp.
435.  ###create list of thetas
436.  THETA = list()
437.
438.  ## generate theta_0 values
439.  ## if you need to do Cholesky with a matrix that is smaller than the Cholesky, multiply by
    Identity matrix then you can multiply by Cholesky.
440.
441.  THETA[[1]] = B
442.       ## Next we create the loop to calculate all THETAS up to T-1
443.  for (i in 1:(TT-1)) {
444.    THETA[[i+1]] = (J%*%An.list[[i]]%*%t(J))%*%THETA[[1]]
445.  }
446.
447.  NAME = colnames(daseas)
448.
449.  # Historical Decomposition
450.  tg = 1
451.  ```
452.
453.  ```{r}
454.  ##PLOTTING HISTORICAL DECOMP
455.  yhat = matrix(NA, TT, 7, dimnames =
456.                 list(c(), c("Shock1", "Shock2", "Shock3", "Shock4", "Shock 5", "Shock 6",
    "Shock 7")))
```

```
457.
458.  tg = 1
459.
460.  ## calculate the first time period manually
461.  tt = 1
462.  yhat[tt,] = THETA[[1]][tg,]%*%diag(w[tt,]) #do all 4 shocks at once
463.
464.
465.  ##calculate all 4 shocks at once for all remaining time periods
466.
467.  ###storing all of the shocks from the yhats across all of the first periods. Slide 15 of
    Lecture 10
468.  for (tt in 2:TT){
469.    jy = THETA[[1]][tg,]%*%diag(w[tt,]) #do all 4 shocks at once
470.    for (i in 1:(tt-1)){
471.      jy = jy + THETA[[i+1]][tg,]%*%diag(w[(tt-i),]) #do all 4 shocks at once
472.    }
473.    yhat[tt,] = jy
474.  }
475.  #sums all rows of each yhat
476.  yyhat = rowSums(yhat) ## all contributions of shocks at each time point. All 4 shocks
    combined at each point in time to movements of Y_t
477.  yhat2 = yhat[,-c(dseasCO)]
478.
479.  ```
480.
481.  ```{r 1bplots}
482.  #plots
483.  par(mfrow = c(1,1))
484.  mi = min(yhat)
485.  mx = max(daseas[-1, tg])
486.
487.  ##homework 7 we identified a VAR(2) model based off lag length, so we start in Q3.
    Historical decomp with each shock and estimated values
488.  plot(ts(yhat[,1], start = c(2015,2), freq = 52),
489.       main = paste("Historical Decomposition for", NAME[tg]), ylab = "", ylim = c(mi, mx))
490.  lines(ts(yyhat, start = c(2015,2), freq = 52), col = "red")
491.  lines(ts(yhat[,2], start = c(2015,2), freq = 52), col = "red")
492.  lines(ts(yhat[,3], start = c(2015,2), freq = 52), col = "blue")
493.  lines(ts(yhat[,4], start = c(2015,2), freq = 52), col = "orange")
494.  lines(ts(yhat[,5], start = c(2015,2), freq = 52), col = "magenta")
495.  lines(ts(yhat[,6], start = c(2015,2), freq = 52), col = "purple")
496.  lines(ts(yhat[,7], start = c(2015,2), freq = 52), col = "dodgerblue")
497.  lines(ts(daseas[-(1:3), tg], start = c(2015,2), freq = 52), col = "green") #we lost two
    data points because of VAR(2) indicates starting 2 lags ahead
498.  legend("bottomleft", legend = c("Shock1", "Shock2", "Shock3", "Shock4", "Shock5",
    "Shock6", "Shock7","Actual"), col = c("black", "red", "blue","orange","magenta", "purple",
    "dodgerblue", "green"), lty = c(1,1,1,1,1,1,1,1), cex = .65)
499.
500.  ```
501.
502.  ```{r 1bplottingY1tn, results = 'hide', fig.show='hide'}
503.  #sum of all shocks with actual data
504.  par(mfrow = c(1,1))
505.  plot(ts(daseas[-1, tg], start = c(2015,2), freq = 52),
506.       main = paste("Historical Decomposition for", NAME[tg]),
507.       ylab = "", ylim = c(min(daseas[-1, tg], yyhat), max(daseas[-(1:4), tg], yyhat)))
508.  lines(ts(yyhat, start = c(2015,2), freq = 52), col = 'dodgerblue')
509.  legend("topleft", legend = c("Actual", "All Shocks"), col = c('black','dodgerblue'),
510.         lty = c(1,1), cex = 0.5)
511.
512.  #The sum of the shocks tracks fairly closely. Look at this without one of the variables
    like CO in future analysis.
513.  ```
```